

ArrayList

Definition: Resizable dynamic array backed by contiguous memory.

How to declare in code: List list = new ArrayList<>();

When to use: Frequent reads, index access, append-heavy workloads.

When to avoid: Frequent insert/delete in middle, memory-sensitive systems.

Algorithms used (names only): Binary Search, Two Pointers, Sliding Window, Prefix Sum

Common Techniques: Capacity pre-sizing, iteration over indexing

---- End ----

LinkedList

Definition: Doubly linked list of nodes.

How to declare in code: List list = new LinkedList<>();

When to use: Frequent insert/remove at ends or via iterators.

When to avoid: Random access, cache-sensitive performance needs.

Algorithms used (names only): BFS (as queue), LRU cache structure

Common Techniques: Use ListIterator, avoid index-based loops

---- End ----

Stack

Definition: LIFO data structure.

How to declare in code: Deque stack = new ArrayDeque<>();

When to use: Recursion simulation, expression parsing, monotonic stack problems.

When to avoid: When deep recursion risk not present or simple loops suffice.

Algorithms used (names only): DFS, Topological Sort, Next Greater Element

Common Techniques: Push-pop patterns, monotonic stack

---- End ----

Queue

Definition: FIFO structure.

How to declare in code: Queue q = new ArrayDeque<>();

When to use: Level-order traversal, task scheduling.

When to avoid: When priority or random access is required.

Algorithms used (names only): BFS, Shortest Path (unweighted)

Common Techniques: Two-queue technique, circular buffering

---- End ----

PriorityQueue (Heap)

Definition: Binary heap based priority structure.

How to declare in code: `PriorityQueue pq = new PriorityQueue<>();`

When to use: Top-K, scheduling, Dijkstra.

When to avoid: When frequent arbitrary removals needed.

Algorithms used (names only): Dijkstra, Prim, Heap Sort

Common Techniques: Custom comparators, lazy deletion

---- End ----

HashMap

Definition: Hash table based key-value store.

How to declare in code: `Map map = new HashMap<>();`

When to use: Fast lookups, counting, caching.

When to avoid: Ordered traversal or range queries required.

Algorithms used (names only): Frequency Count, Two Sum, Memoization

Common Techniques: Load factor tuning, `computeIfAbsent`

---- End ----

TreeMap

Definition: Red-black tree sorted map.

How to declare in code: `Map map = new TreeMap<>();`

When to use: Range queries, ordered keys.

When to avoid: When constant-time lookups are required.

Algorithms used (names only): Sweep Line, Interval Problems

Common Techniques: `floorKey`, `ceilingKey` usage

---- End ----

HashSet

Definition: Hash-based unique collection.

How to declare in code: `Set set = new HashSet<>();`

When to use: Duplicate detection, membership tests.

When to avoid: When order or sorting required.

Algorithms used (names only): Cycle detection, visited tracking

Common Techniques: Use for $O(1)$ membership checks

---- End ----

TreeSet

Definition: Sorted set using red-black tree.

How to declare in code: `Set set = new TreeSet<>();`

When to use: Ordered unique elements, range ops.

When to avoid: When high insertion speed needed.

Algorithms used (names only): Sliding window with ordering

Common Techniques: floor, ceiling operations

---- End ----

Deque

Definition: Double-ended queue.

How to declare in code: `Deque dq = new ArrayDeque<>();`

When to use: Monotonic queue, sliding window max/min.

When to avoid: When random access required.

Algorithms used (names only): 0-1 BFS, Sliding Window

Common Techniques: Maintain increasing/decreasing order

---- End ----

Trie

Definition: Prefix tree for strings.

How to declare in code: `class TrieNode { TrieNode[] next = new TrieNode[26]; boolean end; }`

When to use: Prefix search, autocomplete, dictionary problems.

When to avoid: When memory is limited.

Algorithms used (names only): Word Search, Auto-suggest

Common Techniques: Bitwise trie for XOR problems

---- End ----

Disjoint Set (Union-Find)

Definition: Tracks connected components.

How to declare in code: `class DSU { int[] parent, rank; }`

When to use: Dynamic connectivity, cycle detection.

When to avoid: When graph fully static and small.

Algorithms used (names only): Kruskal, Connected Components

Common Techniques: Path compression, union by rank

---- End ----

Segment Tree

Definition: Binary tree for range queries.

How to declare in code: `class SegTree { int[] tree; }`

When to use: Range sum/min/max with updates.

When to avoid: When data static or small.

Algorithms used (names only): Range Query Problems

Common Techniques: Lazy propagation

---- End ----

Fenwick Tree (BIT)

Definition: Binary indexed tree for prefix sums.

How to declare in code: `class BIT { int[] bit; }`

When to use: Efficient prefix sums, inversion count.

When to avoid: Complex range updates needed.

Algorithms used (names only): Inversion Count

Common Techniques: Lowbit manipulation

---- End ----